

Backend Engineering Intern Case Study Submission

Applicant: Shubham Ganesh Nikumbh

Date: Sept 18, 2025

Role: Backend Intern

Repository: [GitHub Repo Link](#)

Table of Contents:

- 1. Introduction
- 2. Part 1: Code Review & Debugging
- 3. Part 2: Database Design
- 4. Part 3: API Implementation

Introduction:

This document outlines the comprehensive solutions to the Backend Developer Case Study for the Backend Intern position at Bynry Inc. The case study focuses on the development of "StockFlow," a B2B inventory management platform.

The responses are structured into three key sections — Code Review & Debugging, Database Design, and API Implementation — each crafted to demonstrate technical proficiency, attention to real-world business logic, and thoughtful engineering decisions. The proposed solutions aim to balance correctness, scalability, and maintainability while addressing potential edge cases and ambiguities in the requirements.

Part 1: Code Review & Debugging:

1. Identified Issues & Production Impact:

This section addresses the issues found in the provided API endpoint for adding new products.

Issue	Problem	Impact
1. Lack of Transactional Integrity	The code performs two separate <code>db.session.commit()</code> calls. If the second commit fails, the product will be created without its inventory record.	Leads to data inconsistency by creating an orphan product with no stock, making it unmanageable and unsellable.
2. No Data Validation or Error Handling	Data is accessed directly using <code>data['key']</code> without checking for presence or validity. Missing fields will raise <code>KeyError</code> .	API becomes fragile and unreliable. Malformed requests result in unhandled 500 errors, degrading user experience.
3. Non-Enforcement of Business Logic	SKUs are required to be unique but the code doesn't check for duplicates before inserting.	Allows duplicate SKUs, leading to confusion in stock tracking, reporting errors, and fulfillment issues.
4. Missing "Company" Context	The product creation logic lacks any reference to the company creating the product in a multi-tenant environment.	Products are not linked to a business, rendering them unusable in a B2B platform context.

2. Corrected Code & Explanations: (Python + Flask + SQLAlchemy)

```
1 from flask import request, jsonify
2 from sqlalchemy.exc import IntegrityError
3
4 @app.route('/api/products', methods=['POST'])
5 def create_product():
6     """
7     Creates a new product and adds its initial inventory to a specified warehouse.
8     Ensures transactional integrity and validates incoming data.
9     """
10    data = request.get_json()
11
12    # 1. Data Validation: Ensuring all required fields are present and have valid types.
13    required_fields = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity', 'company_id']
14    if not all(field in data for field in required_fields):
15        return jsonify({"error": "Missing required fields"}), 400
16
17    if not isinstance(data.get('price'), (int, float)) or data['price'] < 0:
18        return jsonify({"error": "Price must be a non-negative number"}), 400
19
20    if not isinstance(data.get('initial_quantity'), int) or data['initial_quantity'] < 0:
21        return jsonify({"error": "Initial quantity must be a non-negative integer"}), 400
22
23    try:
24        # 2. Business Logic: Check for SKU uniqueness within the company.
25        existing_product = Product.query.filter_by(sku=data['sku'], company_id=data['company_id']).first()
26        if existing_product:
27            return jsonify({"error": f"SKU '{data['sku']}' already exists"}), 409 # Conflict
28
29        # 3. Atomic Transaction: a single transaction for all database operations.
30        product = Product(
31            name=data['name'],
32            sku=data['sku'],
33            price=data['price'],
34            company_id=data['company_id']
35        )
36        db.session.add(product)
37        db.session.flush() # Flush to get the new product.id before commit.
38
39        inventory = Inventory(
40            product_id=product.id,
41            warehouse_id=data['warehouse_id'],
42            quantity=data['initial_quantity']
43        )
44        db.session.add(inventory)
45
46        db.session.commit() # Single commit for both operations.
47
48        return jsonify({
49            "message": "Product created successfully",
50            "product_id": product.id
51        }), 201 # 201 Created for successful resource creation.
52
53    except IntegrityError:
54        # 4. Error Handling: Rollback transaction on database errors.
55        db.session.rollback()
56        return jsonify({"error": "Database integrity error. Check warehouse_id or other constraints."}), 400
57
58    except Exception as e:
59        db.session.rollback()
60        # to log the error `e` here.
61        return jsonify({"error": "An unexpected error occurred."}), 500
```

Part 2: Database Design:

This section outlines a proposed database schema to meet the system requirements.

1. Proposed Database Schema (SQL DDL)

```
1  -- Stores information about the businesses using the platform.
2  CREATE TABLE Companies (
3      id INT PRIMARY KEY AUTO_INCREMENT,
4      name VARCHAR(255) NOT NULL,
5      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6  );
7
8  -- Each warehouse belongs to a specific company.
9  CREATE TABLE Warehouses (
10     id INT PRIMARY KEY AUTO_INCREMENT,
11     company_id INT NOT NULL,
12     name VARCHAR(255) NOT NULL,
13     location TEXT,
14     FOREIGN KEY (company_id) REFERENCES Companies(id) ON DELETE CASCADE
15 );
16
17 -- Stores supplier information.
18 CREATE TABLE Suppliers (
19     id INT PRIMARY KEY AUTO_INCREMENT,
20     name VARCHAR(255) NOT NULL,
21     contact_email VARCHAR(255),
22     contact_phone VARCHAR(50)
23 );
24
25 -- Manages product-specific low stock thresholds.
26 CREATE TABLE ProductTypes (
27     id INT PRIMARY KEY AUTO_INCREMENT,
28     name VARCHAR(100) NOT NULL UNIQUE,
29     low_stock_threshold INT NOT NULL DEFAULT 20
30 );
31
32 -- Core product information. SKUs are unique per company.
33 CREATE TABLE Products (
34     id INT PRIMARY KEY AUTO_INCREMENT,
35     company_id INT NOT NULL,
36     product_type_id INT,
37     supplier_id INT,
38     name VARCHAR(255) NOT NULL,
39     sku VARCHAR(100) NOT NULL,
40     price DECIMAL(10, 2) NOT NULL,
41     FOREIGN KEY (company_id) REFERENCES Companies(id) ON DELETE CASCADE,
42     FOREIGN KEY (product_type_id) REFERENCES ProductTypes(id),
43     FOREIGN KEY (supplier_id) REFERENCES Suppliers(id) ON DELETE SET NULL,
44     UNIQUE (company_id, sku)
45 );
46
47 -- Tracks product stock in warehouses.
48 CREATE TABLE Inventory (
49     id INT PRIMARY KEY AUTO_INCREMENT,
50     product_id INT NOT NULL,
51     warehouse_id INT NOT NULL,
52     quantity INT NOT NULL CHECK (quantity >= 0),
53     last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
54     FOREIGN KEY (product_id) REFERENCES Products(id) ON DELETE CASCADE,
55     FOREIGN KEY (warehouse_id) REFERENCES Warehouses(id) ON DELETE CASCADE,
56     UNIQUE (product_id, warehouse_id)
57 );
58
59 -- Tracks changes to inventory levels over time for auditing and forecasting.
60 CREATE TABLE InventoryHistory (
61     id INT PRIMARY KEY AUTO_INCREMENT,
62     inventory_id INT NOT NULL,
63     change_amount INT NOT NULL,
64     new_quantity INT NOT NULL,
65     reason VARCHAR(255), -- e.g., 'sale', 'restock'
66     change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
67     FOREIGN KEY (inventory_id) REFERENCES Inventory(id) ON DELETE CASCADE
68 );
69
70 -- Defines which products make up a bundle.
71 CREATE TABLE Bundles (
72     bundle_product_id INT NOT NULL,
73     contained_product_id INT NOT NULL,
74     quantity INT NOT NULL,
75     PRIMARY KEY (bundle_product_id, contained_product_id),
76     FOREIGN KEY (bundle_product_id) REFERENCES Products(id) ON DELETE CASCADE,
77     FOREIGN KEY (contained_product_id) REFERENCES Products(id) ON DELETE CASCADE
78 );
```

2. Gaps and Questions for the Product Team:

- **Product Bundles:** How should a bundle's stock be determined? Is it pre-assembled with its own quantity, or is its availability calculated dynamically from the stock of its component parts?
- **Suppliers:** Can one product be sourced from multiple suppliers? If so, a many-to-many relationship (e.g., a Product_Suppliers table) would be required.
- **Low Stock Alerts:** What is the precise definition of "recent sales activity"? Is it activity within the last 7 days, 30 days, or another period?
- **SKU Uniqueness:** The requirement says SKUs must be "unique across the platform". Does this mean globally unique, or unique per company? I have designed for per-company uniqueness as it is more scalable for a B2B platform.

3. Justification of Design Decisions:

- **Normalization:** The schema separates entities like Companies, Warehouses, and Products to reduce data redundancy and improve data integrity.
- **Indexes:** Primary keys, foreign keys, and unique constraints are indexed to ensure fast lookups and joins, which is crucial for performance.
- **Constraints:**
 - FOREIGN KEY with ON DELETE CASCADE ensures that when a company is deleted, all its dependent data (warehouses, products) is also removed, preventing orphaned records.
 - UNIQUE (company_id, sku) enforces the critical business rule that SKUs must be unique within a company's product list.
- **InventoryHistory Table:** Added to satisfy the requirement to "track when inventory levels change". This is essential for auditing, reporting, and forecasting.

Part 3: API Implementation:

This section provides an implementation for the low-stock alert endpoint.

1. API Implementation (Python/Flask):

Endpoint: GET /api/companies/{company_id}/alerts/low-stock

```
1  from flask import jsonify
2  from datetime import datetime, timedelta
3
4  @app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
5  def get_low_stock_alerts(company_id):
6      """
7      Returns a list of products that are low on stock for a given company,
8      filtered by recent sales activity.
9      """
10     # 1. Edge Case: Handle non-existent company.
11     if not Company.query.get(company_id):
12         return jsonify({"error": "Company not found"}), 404
13
14     # 2. Define recency period for sales.
15     recency_period = datetime.utcnow() - timedelta(days=30)
16
17     try:
18         # 3. Query to find products that are below their stock threshold.
19         low_stock_items = db.session.query(
20             Product.id.label('product_id'),
21             Product.name.label('product_name'),
22             Product.sku,
23             Warehouse.id.label('warehouse_id'),
24             Warehouse.name.label('warehouse_name'),
25             Inventory.id.label('inventory_id'),
26             Inventory.quantity.label('current_stock'),
27             ProductType.low_stock_threshold.label('threshold'),
28             Supplier
29         ).join(Inventory, Product.id == Inventory.product_id)\
30         .join(Warehouse, Inventory.warehouse_id == Warehouse.id)\
31         .join(ProductType, Product.product_type_id == ProductType.id)\
32         .outerjoin(Supplier, Product.supplier_id == Supplier.id)\
33         .filter(
34             Product.company_id == company_id,
35             Inventory.quantity < ProductType.low_stock_threshold
36         ).all()
37
38         alerts = []
39         # 4. Filter results based on recent sales activity.
40         for item in low_stock_items:
41             recent_sale = db.session.query(InventoryHistory.id).filter(
42                 InventoryHistory.inventory_id == item.inventory_id,
43                 InventoryHistory.change_amount < 0,
44                 InventoryHistory.change_date >= recency_period
45             ).first()
46
47             if recent_sale:
48                 supplier_info = None
49                 if item.Supplier:
50                     supplier_info = {
51                         "id": item.Supplier.id,
52                         "name": item.Supplier.name,
53                         "contact_email": item.Supplier.contact_email
54                     }
```

```

55
56         alerts.append({
57             "product_id": item.product_id,
58             "product_name": item.product_name,
59             "sku": item.sku,
60             "warehouse_id": item.warehouse_id,
61             "warehouse_name": item.warehouse_name,
62             "current_stock": item.current_stock,
63             "threshold": item.threshold,
64             "days_until_stockout": 12, # Placeholder value
65             "supplier": supplier_info
66         })
67
68     return jsonify({
69         "alerts": alerts,
70         "total_alerts": len(alerts)
71     }), 200
72
73 except Exception as e:
74     # Log error `e` for debugging.
75     return jsonify({"error": "An error occurred while processing your request."}), 500

```

Sample Response from API:



```

1  {
2    "alerts": [
3      {
4        "product_id": 101,
5        "product_name": "Premium Steel Rod",
6        "sku": "STEEL-ROD-001",
7        "warehouse_id": 201,
8        "warehouse_name": "Pune Central Warehouse",
9        "current_stock": 8,
10       "threshold": 25,
11       "days_until_stockout": 14,
12       "supplier": {
13         "id": 301,
14         "name": "Maharashtra Metals Pvt Ltd",
15         "contact_email": "sales@mhmetals.com"
16       }
17     },

```

2. Assumptions:

- The database schema is the one designed in Part 2.
- "Recent sales activity" is defined as any sale (a negative stock change) recorded in the InventoryHistory table within the last 30 days.
- The days_until_stockout calculation is complex and requires sales velocity analysis. It is included in the response as a placeholder value.

3. Handling Edge Cases & Explanation

- **No Such Company:** The code first validates the company_id to avoid unnecessary work and provides a clear 404 Not Found error.
- **Product without a Supplier:** The query uses an outerjoin to the Suppliers table. This ensures products are included even if they lack a supplier. The code then conditionally builds the supplier object in the final response.
- **Performance:** For very large datasets, checking InventoryHistory inside a loop could be inefficient (N+1 problem). A more optimized solution would use a subquery or a single, more complex JOIN to filter for items with recent sales in one database query.
- **Readability:** The approach separates the initial data fetching from the business logic filtering (recent sales check). This improves code readability and makes it easier to modify the logic for "recent activity" in the future without changing the main database query.